

Chapter 4

Decision Control statement

Decision Control statement

There are methods used in 'C' to control the flow of the program execution namely: **Selection**.

- This involves a decision-making condition to see whether a particular condition is satisfied or not.
- **Selection:** The selection means choosing one among two or more possible cases of actions based on condition using different operators. It is generally done using an **if-else** statement.

If Selection

- If
- If else and nested if
- Switch case

1.The *if* Condition

The statement is executed only when the condition is true. In case the condition is false the compiler skips the line within the if block.

- Syntax for if statement

```
if (condition) /*no semi-colon*/  
    statement;
```

EXAMPLE1

```
#include <stdio.h>
main()
{
    int b;
    printf("Enter a value:");
    scanf("%d", &b);
    if (b < 0)
        printf("The value is negative\n");
}
```

example2

- Write a program to check equivalence of two numbers. Use if statement.

```
#include<stdio.h>
```

```
main(){
```

```
int m,n;
```

```
printf("\n Enter two numbers :");
```

```
scanf("%d %d",&m,&n);
```

```
if(m-n==0)
```

```
printf("\n Two numbers are equal");
```

```
}
```

2. if-else Statement

- The **if....else** statement takes care of true as well as false conditions. It has two blocks. One block is for **if** and it is executed when the condition is true and the other block of **else** is executed when the condition is false.

Below is the syntax of *if-else* statement:

```
if(condition)
{
    /* code block of if statement */
}
else{
    /* code block of else statement */
}
```

example1

- Write a program to check equivalence of two numbers. Use if...else statement.

```
#include<stdio.h>
```

```
main(){
```

```
int m,n,o;
```

```
printf("\n Enter two numbers :");
```

```
scanf("%d %d",&m,&n);
```

```
if(m-n==0)
```

```
printf("\n Two numbers are equal");
```

```
else
```

```
printf("\n Two numbers are not equal");
```

```
}
```

3. else-if(nested *if-else*)

- In the case of many logical conditions, nested *if-else* function is used.
- **The syntax of the if-else-if statement is as follows:**

```
if(condition-1)
{
    /* code block if condition-1 is true */
}
else if (condition-2)
{
    /* code block if condition-2 is true */
}
else if (condition-3)
{
    /* code block if condition-3 is true */
}
else
{
    /* code block all conditions above are false */
}
```

Cont...

- The *else-if* statement is used as the most general way to code multi-way decisions.
- The conditions (condition-1, condition2 ...etc.) are evaluated in sequence from top to bottom.
- If a *condition* is true, the statement or code block associated with it is executed, and this terminates the whole chain.
- If the *else-if* have many conditions, it is good practice to use the [C switch statement](#).
- The last *else* handles the "none of above" or default case where none of the conditions is met.

Here's slightly more complex example:

```
#include<stdio.h>
main()
int b;
printf("Enter the value of b:");
scanf("%d",&b);
if(b<0)
{
printf("The value of b is negative \n");
}
else if(b==0)
{
printf("The value of b is zero\n");
}
else
{
printf("the value of b is Positive\n");
}

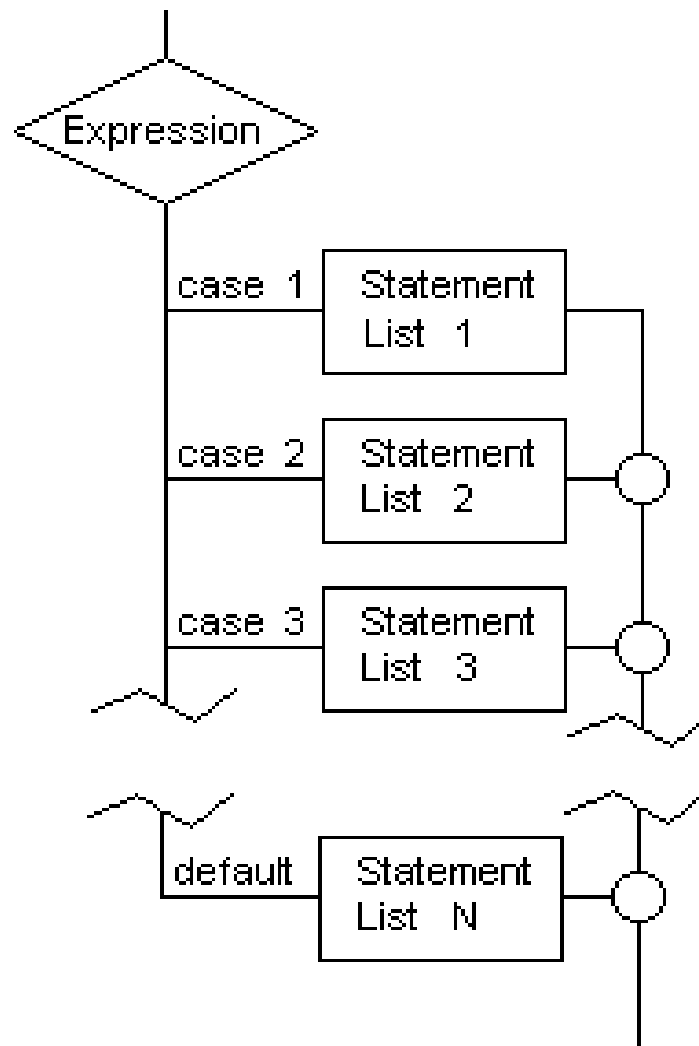
}
```

- #include <stdio.h>
- main()
- { int myBirthday = 13;
- int guess;
- printf("Please guess the day of my birth, from 1 to 31\n");
- printf("Enter your guess, please ");
- scanf("%d",&guess);
- if (guess <= 0) /* Test for out of range guesses first */
- {
- printf("Months have at least one day, Einstein\n");
- }
- else if (guess > 31)
- {
- printf("Pretty long month, genius\n");
- }
- else if (guess == myBirthday)
- {
- printf("Incredible, you are correct\n");
- }
- else if (guess < myBirthday)
- {
- printf("Higher, try again\n");
- }
- else /* Guess is in range and not greater than or equal */
- {
- printf("Lower, try again\n");
- }}

4. *c switch* statement

- *C switch* statement is a mutiway decisions that tests whether a variable or expression matches one of a number of constant integer values, and branches accordingly.
- The switch case statement is a better way of writing a program when a series of if else occurs.

This is the flow chart of the *C switch statement*:



Switch statement

- The following illustrates the C switch statement syntax:

```
switch (expression) {  
    case value1: /* execute unit of code 1 */  
        break;  
    case value2: /* execute unit of code 2 */  
        break;  
    ...  
    default: /* execute default action */  
        break;  
}
```

Rules for switch statements

- Values for 'case' must be integer or character constants.
- The order of the 'case' statements is unimportant.
- The default clause may occur first (convention places it last)

cont

- The keyword **break** allows the programmers to terminate the loop. The **break** skips from the loop or block in which it is defined. The control then automatically goes to the first statement after the loop or block.
- The *break* statement is not mandatory, but if you don't put a *break* statement at the end of the statements for a case, the statements for the next case in sequence will be executed as well, through to whenever another break is found or the end of the switch block is reached. This can lead some unexpected program logic happens.

Let's examine the C switch statement syntax in more detail:

- In C *switch* statement, the selection is determined by the value of an expression that you specify, which is enclosed between the parentheses after the keyword *switch*. The data type of value, which is returned by expression, must be an integer value otherwise the compiler will issue an error message.
- The case constant expression must be a constant integer value.
- When a *break* statement is executed, it causes an immediate exit from the *switch*. The control pass to the statement following the closing brace for the switch.
- The *default* statement is the default choice of the switch statement if all case statements are not satisfied with the expression. The *break* after the default statements is not necessary unless you put another case statement below it.

Example of C Switch Statement

```
#include <stdio.h>
```

```
main() {
```

```
    int color ;
```

```
    printf("Please choose a color(1: red,2: green,3: blue):\n");
```

```
    scanf("%d", &color);
```

```
    switch (color) {
```

```
        case 1:
```

```
            printf("you chose red color\n");
```

```
            break;
```

```
        case 2:
```

```
            printf("you chose green color\n");
```

```
            break;
```

```
        case 3:
```

```
            printf("you chose blue color\n");
```

```
            break;
```

```
        default:
```

```
            printf("you did not choose any color\n");
```

```
    } return 0;}
```